

DATA ENGINEERING COURSE

NumPy

Numerical Python

Student Assignment

Class Assignment + Home Assignment

Instructor: Tanishq Tyagi

Data Engineering Course

Student Name:	_____	Roll No:	_____
Date:	_____	Batch:	_____

PART A — Class Assignment

Student Performance Analyzer

A teacher has recorded the marks of **8 students**. Create a NumPy array of **8 random integer marks** (between 40 and 100) and complete all the tasks below.

*** Use only NumPy operations — do NOT use Python loops unless the task explicitly requires it.**

Task 1 — Create & Print the Marks Array

Create a NumPy array of 8 random integer marks using a list comprehension and **np.array()**. Print the array.

Write your code here:

```
import numpy as np
import random

marks = np.array([random.randint(40, 100) for _ in range(8)])
print('Marks:', marks)
```

Expected output format:

```
Marks: [xx xx xx xx xx xx xx xx]
```

Task 2 — Array Attributes

Using the **marks** array created above, print all four attributes.

Attribute	Code to Write	Expected Output
Shape	<code>marks.shape</code>	<code>(8,)</code>
Size	<code>marks.size</code>	<code>8</code>
Dimensions	<code>marks.ndim</code>	<code>1</code>
Data type	<code>marks.dtype</code>	<code>int64</code>

Task 3 — Statistical Summary

Calculate and print the following statistics for the marks array:

Statistic	NumPy Method	Description
Total marks	<code>marks.sum()</code>	Sum of all 8 marks
Average marks	<code>marks.mean()</code>	Arithmetic mean
Highest marks	<code>marks.max()</code>	Maximum value
Lowest marks	<code>marks.min()</code>	Minimum value

Task 4 — Grace Marks (Broadcasting)

Add **5 grace marks** to every student's score using NumPy broadcasting (no loops allowed). Store the result in **marks_grace** and print it.

```
marks_grace = marks + 5
print('After grace marks:', marks_grace)
```

*** Broadcasting tip:** NumPy automatically applies the scalar +5 to every element in a single C-level operation. No for loop needed.

Task 5 — 10% Marks Increase

Increase every student's original marks by **10%** using broadcasting. Store in **marks_raised** and print.

```
marks_raised = marks * 1.10
print('After 10% raise:', marks_raised)
```

Task 6 — Special Arrays

Create and print the following three special arrays:

Array	Function	Expected Output
8 zeros	<code>np.zeros(8, dtype=int)</code>	<code>[0 0 0 0 0 0 0 0]</code>
8 ones	<code>np.ones(8, dtype=int)</code>	<code>[1 1 1 1 1 1 1 1]</code>
Roll nos 1–8	<code>np.arange(1, 9)</code>	<code>[1 2 3 4 5 6 7 8]</code>

PART B — Home Assignment

Supermarket Sales Analyzer

A supermarket recorded its daily sales (in ₹) for **10 consecutive days**. Use the pre-defined array below and complete all tasks.

```
import numpy as np

sales = np.array([1250, 1380, 1190, 1500, 1425,
1600, 1550, 1490, 1700, 1650])
```

! Submit this assignment before the next class. Show all code and output.

Task 1 — Print the Sales Array

Print the **sales** array and verify it has 10 elements.

Your output:

Task 2 — Array Attributes

Display the **shape**, **size**, **number of dimensions**, and **data type** of the sales array. Fill in the expected values:

Attribute	Code	Your Output
Shape	sales.shape	
Size	sales.size	
Dimensions	sales.ndim	
Data type	sales.dtype	

Task 3 — Statistical Summary

Calculate and record the following statistics:

Statistic	NumPy Method	Your Answer
Total sales	<code>sales.sum()</code>	■ _____
Average daily	<code>sales.mean()</code>	■ _____
Highest sales	<code>sales.max()</code>	■ _____
Lowest sales	<code>sales.min()</code>	■ _____

Task 4 — Add ■100 Using Broadcasting

Increase every day's sales by ■100 using broadcasting. Store in **sales_plus100** and print.

Write your code:

Your output:

Task 5 — 10% Sales Increase

Calculate the sales after a **10% increase** using broadcasting. Store in **sales_raised** and print.

Write your code:

Your output:

Task 6 — Special Arrays

Create and print the following arrays:

Array	Function to Use	Your Output
10 zeros (returned)	<code>np.zeros(10, dtype=int)</code>	
10 ones (status)	<code>np.ones(10, dtype=int)</code>	

Day numbers 1–10	<code>np.arange(1, 11)</code>	
------------------	-------------------------------	--

Task 7 — Performance Benchmark

Compare the execution time of multiplying **one million numbers by 2** using (a) a Python list with a for loop and (b) a NumPy array. Use **`time.time()`** to measure and print both times.

Starter code structure:

```
import numpy as np
import time

numbers = list(range(1_000_000)) # Python list
array = np.arange(1_000_000) # NumPy array

# --- Python List ---
result = []
start = time.time()
# YOUR CODE HERE
list_time = time.time() - start

# --- NumPy ---
start = time.time()
# YOUR CODE HERE
numpy_time = time.time() - start

print(f'List time : {list_time:.5f} seconds')
print(f'NumPy time: {numpy_time:.5f} seconds')
```

Record your execution times:

Method	Execution Time (seconds)
Python List	
NumPy Array	
Speed-up ratio (List / NumPy)	

Explanation — In 2–3 lines, explain which method is faster and why:

.....

.....

.....

Submission Checklist

#	Item	Done
1	Part A — Class Assignment: All 6 tasks completed with code and output	☐
2	Part B — Task 1: Sales array printed	☐
3	Part B — Task 2: All 4 attributes filled in the table	☐
4	Part B — Task 3: All 4 statistics computed and recorded	☐
5	Part B — Task 4: Broadcasting code written and output noted	☐
6	Part B — Task 5: 10% increase code written and output noted	☐
7	Part B — Task 6: All 3 special arrays created and outputs recorded	☐
8	Part B — Task 7: Benchmark times recorded + 2–3 line explanation written	☐